

Validation in Spring Boot — Interview Notes

1. `@Valid`

Triggers validation of a request DTO using Bean Validation annotations.

Java

```
@PostMapping
public User create(@Valid @RequestBody UserRequest request) { }
```

2. Common Validation Annotations

Annotation	Purpose
<code>@NotNull</code>	Value cannot be <code>null</code>
<code>@NotBlank</code>	String cannot be null/empty/whitespace
<code>@NotEmpty</code>	String/Collection cannot be empty
<code>@Size</code>	Validates length/size
<code>@Min</code> , <code>@Max</code>	Numeric range
<code>@Email</code>	Valid email format
<code>@Pattern</code>	Regex validation
<code>@Past</code> , <code>@Future</code>	Date validation

Key: `@NotNull` allows `""`; `@NotBlank` does not.

3. Validation Failure

For:

Java

```
@Valid @RequestBody UserRequest request
```

invalid input commonly results in:

MethodArgumentNotValidException

Handle it globally:

Java

```
@RestControllerAdvice
class GlobalExceptionHandler {

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<?> handleValidation(...) {
        // return field errors
    }
}
```

4. @Valid vs @Validated

- @Valid → standard object validation.
- @Validated → Spring annotation; supports **validation groups** and method-level validation.

5. Custom Validator

Use when built-in annotations aren't enough.

```
Custom Annotation
    ↓
@Constraint(validatedBy = ...)
    ↓
ConstraintValidator
    ↓
isValid()
```

Example:

Java

```
@ValidUsername
private String username;
```

Custom validator:

Java

```
class UsernameValidator
    implements ConstraintValidator<ValidUsername, String> {

    public boolean isValid(String value, ...) {
        return value.matches("...");
    }
}
```

Interview one-liner

`@Valid` triggers Bean Validation, built-in annotations define constraints, and `ConstraintValidator` allows us to implement custom validation rules.